ClariFix implementation alignment addendum

Updated Group 4 paper: lexical analyzer submission

COSC 303: Programming Language Design | BSCS 3-3 | Group 4

Revision: paper-alignment-2. This addendum documents how the analyzer implements the user's updated paper, especially pages 18–20. The authoritative paper is included unchanged as `ClariFix-Updated-Paper.pdf`. This addendum supersedes the earlier implementation proposal's symbol-storage and output decisions; it does not silently rewrite the group's paper.

Group members: Balayanto, Czarina Mae M.; Brase, Mark James B.; Castillo, Marc Lowel J.; Derecho, Jimson Aldrich G.; Hipolito, Brandon Boyd M.; Intong, Sean Gabriel M.; Regencia, Hannah Samantha C.

1. Scope

The current deliverable is a handwritten C11 lexical analyzer. It recognizes lexemes, records locations, interns identifiers/constants, reports lexical errors, and produces the specified Symbol Table text file. Parsing, name/type analysis, and interpretation remain later phases, consistent with page 13 of the paper.

The correct average program on pages 15–16 is a lexical acceptance example. The deliberately wrong program can also pass lexical analysis because missing colons, unknown variables, assignment in conditions, and text/number operations require later phases. Lexical success does not certify a runnable program.

2. Symbol storage: pages 18–19

The symbol table starts empty and uses manually implemented djb2 hashing, 211 fixed buckets, and separate chaining. Hash arithmetic wraps at 64 bits before taking the bucket index modulo 211. The key is the case-sensitive pair of original lexeme and token type. IDs start at 1 and follow first insertion. Repeated occurrences reuse the original entry and increment an occurrence count.

Character automata recognize keywords, noise words, operators, delimiters, and Boolean spellings before symbol insertion. Only identifiers and constants enter the hash table. Reserved words and symbolic operators are not symbol entries. Boolean literals are accepting states outside the reserved array and are inserted as constants only when encountered.

A character trie supplies exact lexeme identities. Each hash bucket compares terminal identity plus token type, using integer comparisons rather than completed-word string matching. Hash collisions therefore retain distinct symbols while meeting the instructor's character-based recognition restriction. The trie assists collision equality; the 211-bucket table performs symbol lookup and storage.

Attribute	Meaning
ID	One-based index, assigned on first insertion.
Lexeme	Original, case-sensitive source spelling.
Token type	Public token class, such as IDENTIFIER or CONST_INT.
Category	Identifier or Constant for symbol rows.
Value	Converted constant value; identifiers use a dash.
Length	Unicode-scalar count of the source lexeme, including quotes/escape spelling.
First location	One-based line and byte column.

Attribute	Meaning
Occurrences	Number of times that exact lexeme/type key occurs.

Numeric constant values use exact normalized base-10 text, avoiding silent machine overflow or rounding at lexical time. Integer leading zeros and redundant decimal fractional zeros are removed; an integral decimal retains `.0`. Strings and characters decode the supported escapes; Boolean values are true or false. Original spellings remain distinct keys even when converted values agree.

3. Report: page 20

The report prints the source basename and four sections, using fixed-width left-aligned columns that expand to fit content. Empty data sections print `(none)`.

Section	Columns or contents
A: Token List	No., Line, Col, Lexeme, Token Type, Category; every successful source occurrence in order.
B: Symbol Table	ID, Lexeme, Token Type, Value, Length, First Seen; unique identifiers/constants only.
C: Invalid Tokens	Line, Col, Lexeme, Error, Reason; offending fragments for reported lexical diagnostics.
D: Summary	Occurrence counts per category, invalid count, and total successful source tokens.

A includes keywords, noise, operators, delimiters, literals, and comments. It excludes synthetic EOF, NEWLINE, INDENT, and DEDENT. B deduplicates names/constants. C includes malformed recognized constructs as well as unknown characters. Recovery-skipped source is not falsely presented as individually analyzed tokens. Invalid fragments are counted separately from successful source tokens.

Control characters and invalid bytes are escaped so each row occupies one physical line. Valid UTF-8 remains intact. The public names follow the sample: IDENTIFIER, CONST_INT, CONST_DECIMAL, CONST_CHAR, CONST_STRING, CONST_BOOLEAN, NOISE, SINGLE_COMMENT, MULTI_COMMENT, and STATEMENT_TERMINATOR. Other keyword/operator/delimiter names remain unchanged.

4. Exact sample acceptance

```
# avg calc
set total to 10.
show total.
```

The source is stored in `examples/sample.clx`. Its A section has nine rows. Its B section contains exactly:

ID	Lexeme	Token Type	Value	Length	First Seen
1	total	IDENTIFIER	-	5	2:5
2	10	CONST_INT	10	2	2:14

C prints `(none)`. D records identifiers 2, keywords 2, noise words 1, constants 1, delimiters 2, comments 1, operators 0, invalid 0, total tokens 9. These expectations are independently asserted in the test suite.

5. Lexical rules retained

Identifiers match ASCII `[A-Za-z_][A-Za-z0-9_]*`, are case-sensitive, and have a 32-character maximum. Integer, decimal, character, string, and Boolean literals are recognized. Arrays are bracket/element/comma syntax rather than a single scalar token. A decimal point requires a following digit; minus remains separate.

Literals use straight quotes and the supported escapes: backslash followed by n, double quote, single quote, or backslash. Character literals contain exactly one UTF-8 scalar or escape. Raw newlines, NUL, invalid UTF-8, unsupported escapes, and smart quote boundaries are rejected. Hash comments and non-nesting slash-star comments are recognized; double slash is floor division.

All listed arithmetic, relational, logical, and assignment operators use longest match. Word aliases and/or/not map to the logical kinds. The word is has its own token. Delimiters include parentheses, square brackets, commas, periods, colons, and braces in explicit braces mode.

6. Decisions for conflicting paper passages

Conflict	Implemented decision
The sample uses .clx; older examples use .clf.	.clx is canonical; .clf remains a compatibility alias.
Page 8 says free-field braces; pages 14–15 use indentation.	Default to four-space indentation; select --braces explicitly for free-field code.
Page 9 uses smart quotes despite the explicit ban.	Follow the straight-quote rule; treat smart quotes in examples as typography mistakes.
Page 3 prints a Unicode dash; page 5 uses two hyphens.	Use ASCII -- for decrement.
Noise is said to be dropped, but assignment requires a separator.	Report original noise; normalize allowed to to ASSIGN for parser tokens; omit other allowed fillers.
Noise misuse is assigned to the parser in one phase table.	Preserve the existing early E105 lexical check and document it.
Optional int declaration is absent from reserved words and grammar.	Keep int as an identifier; use set score = 90. for valid declarations.

These decisions are documented rather than hidden. The original PDF remains unchanged. `updated-paper-decisions.md` gives page-by-page traceability and implementation responsibilities.

7. Diagnostics and outputs

Diagnostics include the source location, what happened, why, and how to fix it. Recovery skips the rest of a damaged physical line and removes that line's unmatched openers. Up to five errors are reported, then scanning stops with Fix these first, then run again. Invalid UTF-8 and control bytes are escaped. Numeric byte-column locations remain authoritative where terminal glyph widths differ.

Exit codes are 0 for lexical success, 1 for lexical errors, and 2 for usage, memory, or I/O failures. Reports after lexical errors are explicitly partial. Missing colons/periods, undefined names, reserved words used as variables, type checks, and runtime behavior remain later phases.

The main report flag is --report FILE. Developer exports remain available: --symbols FILE for unique symbol attributes, --tokens FILE for internal parser tokens, and --table FILE for a simple two-column source listing. Default stdout remains the detailed parser-token stream. Output destinations must not already exist.

8. Build, validation, and hand-in

```
.\build.ps1 -Test
.\build\clarifix.exe --report build\sample-report.txt examples\sample.clx
.\build.ps1 -Examples -OutputDirectory build\fresh-reports
```

Use a fresh output filename/directory. The Windows build uses desktop GCC with strict C11 warnings; an existing terminal can discover a recently installed compiler through the saved PATH. Python is needed for tests and packaging, not lexical recognition.

Tests cover the page-20 sample, hash collisions, duplicate/case-sensitive keys, all constant types and converted values, Unicode lengths, very large numeric values, invalid fragments, recovery, layout modes, extensions, protected exports, and existing token/location regressions. Validation records actual commands and results in `validation.md`.

The source-only ZIP includes the authoritative paper, this printable addendum, source, tests, examples, generated reports, and validation evidence. The manifest is checked, then a clean extraction is compiled/tested and its reports compared with the samples. GitHub release paper-alignment-2 preserves the earlier release as historical. The group must still confirm the actual deadline and submission destination, provide instructor access to the private repository, hand in the ZIP and link, print the paper/addendum, and prepare its index card.